

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 24. Februar 2026

Ertan Ertas

Kaiserslautern, 24. Februar 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen	5
2.1	Typentheorie und Datenmodellierung	5
2.1.1	Dynamische Typisierung in Python (Duck Typing)	5
2.1.2	Statische Typisierung und Control Flow Analysis in TypeScript	6
2.1.3	Type Narrowing und Discriminated Unions	7
2.1.4	Strukturelle Typisierung (Structural Typing)	8
2.2	Algorithmik von Mengen und Abbildungen	10
2.2.1	Identität und Gleichheit (Value Semantics)	10
2.2.2	Hash-Funktionen und Verteilung	10
2.2.3	Kollisionsauflösung: Open Addressing und SoA	11
2.2.4	Löschstrategien: Backshift Deletion	11
2.3	Termersetzung und Pattern Matching	12
2.3.1	Termersetzungssysteme (TRS)	12
2.3.2	Pattern Matching und Unifikation	13
2.3.3	Fixpunkt-Iteration	13
2.4	Formale Sprachen und Syntaxanalyse	13
2.4.1	LR-Parsing und der Concrete Syntax Tree (CST)	13
2.4.2	Transformation zum Abstrakten Syntaxbaum (AST)	14
2.5	Laufzeitumgebungen und WebAssembly	14
3	Lezer als Parser-Generator	16
3.1	Einordnung in die Theorie formaler Sprachen	16
3.2	Lezer: Konzept und Architektur	16
3.3	Grammatikdefinition in Lezer	17
3.4	Parsebäume und abstrakte Syntaxbäume	18
3.5	Inkrementelles Parsing und Komplexität	18
3.6	Bedeutung für die Python–TypeScript–Übersetzung	19
3.7	Zusammenfassung Kapitel 3	19
4	Transformation und Codegenerierung	19
4.1	Einordnung als syntaxgerichtete Übersetzung	20
4.2	Abbildung von Python- auf TypeScript-Typen (erweitert)	20

4.2.1	Grundabbildungen	20
4.2.2	Unions, Literale und Generics (Beispiele aus der Portierung)	21
4.2.3	Schlüsselstrukturen: tuple/frozenset als Keys	21
4.2.4	Automaten: set/frozenset, Transitionen und Zustandsmengen	21
4.2.5	Rekursive Strukturen: ParseTree, RegExp und AST	23
4.2.6	Parser-Rückgabewerte: tuple $\rightarrow$ TS-Tupeltyp	24
4.3	Transformation des abstrakten Syntaxbaums	24
4.4	Behandlung semantischer Unterschiede	24
4.5	Erzeugung von TypeScript-Code	25
4.6	Korrektheit der Transformation	25
4.7	Zusammenfassung Kapitel 4	26
5	Validierung, Tests und Grenzen der Portierung	26
5.1	Statische Absicherung durch den TypeScript-Compiler	26
5.2	Dynamische Validierung durch Tests	27
5.2.1	Unit-Tests für lokale Funktionen	27
5.2.2	Integrations-Tests entlang der Pipeline	27
5.2.3	Differentialtests: Python vs. TypeScript	27
5.3	Typinvarianten als Test	28
5.4	Grenzen und typische Stolperstellen	28
5.5	Zusammenfassung Kapitel 5	28
6	Kritische Reflexion und Ausblick	29
6.1	Erreichte Ziele und zentrale Ergebnisse	29
6.2	Mehrwert der statischen Typisierung im Lehr- und Projektkontext	29
6.2.1	Frühzeitige Fehlererkennung und bessere Schnittstellen	30
6.2.2	Dokumentation durch Typen und Wartbarkeit	30
6.2.3	Strukturelle Typisierung als Brücke zu Python-Datenmodellen	30
6.3	Einordnung der Transformation als compiler-nahe Übersetzung	30
6.4	Grenzen der Portierung	31
6.5	Ausblick: Technische Erweiterungen und Forschungsfragen	31
6.5.1	(Teil-)Automatisierte Typableitung und Übersetzung	31
6.5.2	Systematische Korrektheitsabsicherung	31
6.5.3	Didaktische Aufbereitung als Toolchain	32
6.5.4	Erweiterung des Sprachumfangs	32
6.6	Schlussbemerkung	32
7	Literaturverzeichnis	33

A	Anhang	34
A.1	Verzeichnis der portierten Lehrnotebooks	34

Abkürzungsverzeichnis

DHBW Duale Hochschule Baden-Württemberg

Abbildungsverzeichnis

1	Gegenüberstellung der Gleichheitssemantik zwischen Python und TypeScript	2
2	Control Flow Analysis: Der TypeScript-Compiler verfeinert (narrowt) den Typ basierend auf logischen Bedingungen.	6
3	Visualisierung von Linear Probing und Clusterbildung. Elemente B und C kollidieren mit A. Element D (Hash 3) trifft auf das belegte Feld 3 und muss auf 4 ausweichen, obwohl es gar nicht Hash 1 hat.	11

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	34
---	---	----

1 Einleitung

Die Vermittlung theoretischer Informatikkonzepte profitiert maßgeblich von der praktischen Umsetzung in ausführbarem Code. An der Dualen Hochschule Baden-Württemberg (DHBW) werden Konzepte wie Tokenisierung, Parsing-Algorithmen und Automatenmodelle anhand von Python-Notebooks demonstriert. Diese Wahl ist didaktisch begründet durch die hohe Lesbarkeit von Python. In einem „compiler-nahen“ Kontext, der sich durch rekursive Datenstrukturen und strikte Zustandsübergänge auszeichnet, offenbaren sich jedoch die Grenzen einer rein dynamisch typisierten Sprache. Parallel dazu hat sich TypeScript als Industriestandard etabliert, der die Flexibilität von Skriptsprachen mit der Sicherheit statischer Typsysteme verbindet.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in das TypeScript-Ökosystem. Der Fokus liegt dabei auf einer idiomatischen Portierung, die nicht nur die Syntax übersetzt, sondern die semantischen Konzepte der formalen Sprachen durch das statische Typsystem explizit modelliert.

1.1 Problemstellung

Die Portierung algorithmischer Kerne von Python nach TypeScript stellt spezifische Anforderungen an die Modellierung der Datenstrukturen und die Fehlererkennung. Folgende Problemfelder sind hierbei zentral:

Grenzen der optionalen Typisierung in Python

Obwohl Python mit Werkzeugen wie `mypy` Möglichkeiten zur statischen Typprüfung bietet, erweisen sich diese im Kontext interaktiver Jupyter-Notebooks oft als unzureichend. In der Praxis führt das dynamische Nachladen von Modulen in der Notebook-Umgebung häufig dazu, dass der Type-Checker inkonsistente Zustände meldet oder deaktiviert werden muss. Zudem fehlt dem Python-Typsystem oft die nötige Granularität bei der Flussanalyse (Control Flow Analysis): Während TypeScript durch „Type Narrowing“ innerhalb eines `if`-Blocks den Typ einer Variable verfeinern kann, muss in Python häufig auf `cast` oder `type: ignore` zurückgegriffen werden, um Validierungsfehler zu unterdrücken. Dies untergräbt das Vertrauen in die Typsicherheit, gerade bei komplexen verschachtelten Strukturen wie abstrakten Syntaxbäumen (AST).

Divergenz der Gleichheitssemantik (Value vs. Reference Semantics)

Ein fundamentales Hindernis bei der direkten Übersetzung ist die unterschiedliche Behandlung von Objektidentität und Gleichheit. Algorithmen der theoretischen Informatik, insbesondere die Potenzmengenkonstruktion oder die Minimierung

von Automaten, basieren auf der Annahme, dass Mengen von Zuständen oder Tupel als Schlüssel in Abbildungen verwendet werden können. In Python gilt: Zwei logisch identische Tupel oder Mengen werden als gleich behandelt (Wertgleichheit) und generieren denselben Hash-Wert. In der JavaScript/TypeScript-Laufzeitumgebung hingegen basieren `Set` und `Map` bei Objekten und Arrays auf Referenzgleichheit. Zwei strukturell identische Objekte an unterschiedlichen Speicheradressen gelten als ungleich.

Abbildung 1 verdeutlicht diese Diskrepanz, die eine native Verwendung von Standard-Datenstrukturen für die genannten Algorithmen verhindert.

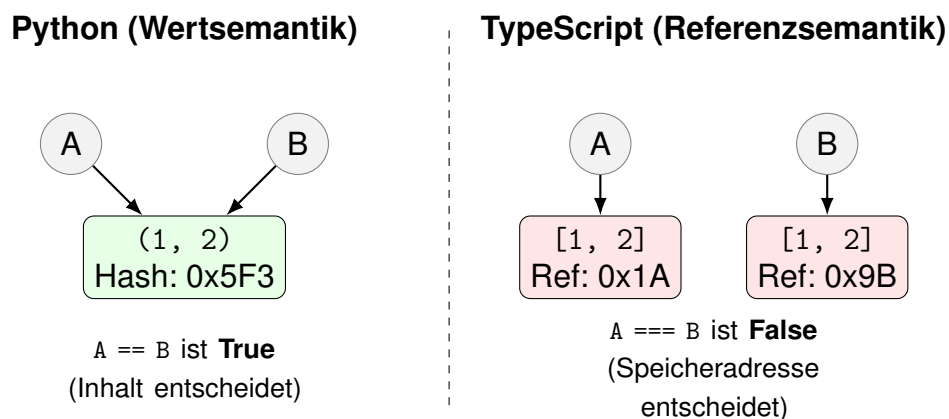


Abbildung 1: Gegenüberstellung der Gleichheitssemantik zwischen Python und TypeScript

Quelle: Eigene Darstellung.

Herausforderungen bei der KI-gestützten Code-Generierung

Der Einsatz von Large Language Models (LLMs) zur Unterstützung der Portierung verspricht zwar eine Beschleunigung des Entwicklungsprozesses, ist jedoch mit Risiken hinsichtlich der semantischen Korrektheit behaftet. So neigen LLMs dazu, Inhalte zu generieren, die auf den ersten Blick plausibel wirken, sich bei genauerer Betrachtung jedoch als faktisch inkorrekt erweisen, ein Phänomen, welches in der Forschung als „Halluzination“ beschrieben wird [1].

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist die Bereitstellung einer vollständig typisierten TypeScript-Implementierung der Lehrinhalte aus der Vorlesung „Theoretische Informatik III“. Das Ergebnis soll eine Codebasis sein, welche die ursprünglichen Python-Notebooks funktional ersetzt, jedoch die algorithmischen Strukturen durch das statische Typsystem formal absichert.

Konkret wird angestrebt, die von der Dozentenseite kritisierte Schwerfälligkeit

der Python-Typisierung durch eine idiomatische TypeScript-Modellierung zu ersetzen. Ziel ist es, nachzuweisen, dass komplexe Invarianten von Syntaxbäumen und Automaten durch moderne Sprachfeatures wie *Discriminated Unions* und *Control Flow Analysis* bereits zur Kompilierzeit garantiert werden können. Die Arbeit soll aufzeigen, an welchen Stellen die statische Analyse von TypeScript der dynamischen Natur von Python in Bezug auf Wartbarkeit und Fehlervermeidung überlegen ist.

Ein wesentliches technisches Teilziel ist die Konzeption und Implementierung einer spezialisierten Bibliothek, welche die Lücke zwischen der Wertsemantik von Python und der Referenzsemantik von JavaScript schließt. Es soll eine performante Lösung entwickelt werden, die echtes Hashing und strukturelle Gleichheit für beliebig verschachtelte Datenstrukturen ermöglicht, um die Algorithmen der Mengenlehre ohne semantische Brüche portieren zu können.

Des Weiteren wird eine hohe technische Interoperabilität angestrebt. Die resultierende Lösung soll plattformunabhängig auf den gängigen Betriebssystemen (Windows, macOS, Linux) ausführbar sein. Durch die Reduktion nativer Abhängigkeiten soll die Einstiegshürde für Studierende minimiert und eine konsistente Ausführungsumgebung gewährleistet werden.

Begleitend verfolgt die Arbeit das methodische Ziel, die Effektivität von Large Language Models in der Software-Modernisierung zu evaluieren. Es soll untersucht werden, inwieweit ein iterativer Prozess die Problematik von Halluzinationen bei komplexen algorithmischen Übersetzungen mitigieren kann.

1.3 Methodik und Abgrenzung

Die Durchführung der Arbeit folgt einem selektiven und adaptiven Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Die technische Umsetzung umfasst nicht nur den algorithmischen Kern, sondern auch die notwendige Infrastruktur zur Visualisierung. Ein zentrales Entwurfsziel war hierbei die Minimierung externer Systemabhängigkeiten, um eine reibungslose Nutzung auf verschiedenen Betriebssystemen zu gewährleisten. Anstatt eine lokale Installation binärer Drittsoftware vorauszusetzen, wurde die Visualisierung durch den Einsatz portabler, direkt in die Laufzeitumgebung integrierbarer Bibliotheken realisiert. Dieser Ansatz stellt sicher, dass die Notebooks

allein durch die Bereitstellung einer *Conda*-Umgebung vollständig und ohne weitere Installationsschritte ausführbar sind.

Ein zentraler methodischer Aspekt betrifft die textuelle Aufbereitung: Die begleitenden Markdown-Erklärungen in den Notebooks wurden nicht bloß übersetzt, sondern redaktionell so überarbeitet, dass sie die TypeScript-spezifischen Konzepte eigenständig und in sich geschlossen erläutern. Auf vergleichende Referenzen zur Python-Implementierung wurde im Fließtext bewusst verzichtet, um die didaktische Eigenständigkeit der TypeScript-Version zu wahren.

Entsprechend der Anforderung an die Reproduzierbarkeit und Zugänglichkeit der Ergebnisse wird der gesamte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstreh/rath/formal-languages-typescript> im Ordner TypeScript verfügbar.

2 Theoretische Grundlagen

Dieses Kapitel legt das theoretische Fundament für die Transformationsstrategie. Ein besonderer Schwerpunkt liegt auf der Analyse der Typsysteme, da die Diskrepanz zwischen der dynamischen Natur von Python und der statischen Analyse von TypeScript die zentrale Motivation für die Portierung darstellt.

2.1 Typentheorie und Datenmodellierung

Ein Typsystem ist ein formales logisches System, das Programmkonstrukten Eigenschaften (Typen) zuweist, um das Programmverhalten zu klassifizieren. Nach Pierce gilt ein Programm als typsicher, wenn wohlgetypte Programme nicht in einen undefinierten Zustand geraten („well-typed programs do not go wrong“) [2, S.1ff].

In der Typentheorie wird Typsicherheit häufig durch zwei Eigenschaften charakterisiert [2, S.95ff]:

- **Progress:** Ein wohlgetypter Ausdruck ist entweder bereits ein Wert oder kann einen weiteren Auswertungsschritt durchführen.
- **Preservation:** Wenn ein wohlgetypter Ausdruck einen Auswertungsschritt macht, behält das Resultat diesen Typ bei.

2.1.1 Dynamische Typisierung in Python (Duck Typing)

Python folgt dem Paradigma der dynamischen Typisierung. Typen sind hierbei Eigenschaften von Werten (Laufzeitobjekten), nicht von Variablen. Eine Variable ist lediglich ein Name, der an ein beliebiges Objekt gebunden werden kann. Die Typprüfung erfolgt implizit zur Laufzeit: Eine Operation (z. B. `obj.method()`) ist zulässig, wenn das Objekt zum Zeitpunkt des Aufrufs über das entsprechende Attribut verfügt.

Dieses Konzept wird als **Duck Typing** bezeichnet („If it walks like a duck and quacks like a duck, it is a duck“). Während dies maximale Flexibilität und schnelle Prototypenentwicklung ermöglicht, birgt es im Kontext compiler-naher Softwarearchitekturen signifikante Risiken:

1. **Späte Fehlererkennung:** Inkonsistenzen in Datenstrukturen (z. B. ein fehlendes Feld in einem AST-Knoten) führen erst dann zu einem `AttributeError`, wenn der entsprechende Codepfad zur Laufzeit durchlaufen wird.
2. **Implizite Annahmen:** Die Struktur komplexer Objekte (z. B. „Ein Token ist ein Tupel aus Typ und Wert“) muss durch Dokumentation oder Konvention gesichert werden, da die Sprache sie nicht erzwingt.

Zwar bietet Python mit `mypy` ein Werkzeug für optionale statische Typannotationen (PEP 484), dessen Analysefähigkeit ist jedoch begrenzt. Insbesondere bei der Flussanalyse (Control Flow) kann `mypy` oft nicht automatisch deduzieren, dass ein Typ nach einer Prüfung sicher ist, was zu erzwungenen `cast`-Operationen oder `# type: ignore`-Kommentaren führt.

2.1.2 Statische Typisierung und Control Flow Analysis in TypeScript

Im Gegensatz dazu führt TypeScript eine strikte statische Typprüfung vor der Ausführung durch. Ein zentrales Unterscheidungsmerkmal zu traditionellen statischen Sprachen (wie Java) ist dabei die Fähigkeit zur **Control Flow Analysis (CFA)**. Der Compiler analysiert den logischen Fluss des Codes, um den Typ einer Variable an jeder Stelle im Code so präzise wie möglich zu bestimmen.

Dieses Verfahren wird als **Type Narrowing** bezeichnet. Aus einem breiten Typ (z. B. `string | number`) wird durch Bedingungen ein engerer Typ gefiltert [3].

Abbildung 2 visualisiert diesen Prozess: Der Compiler „versteht“ die `if`-Bedingung und garantiert im jeweiligen Zweig den korrekten Typ, ohne dass zur Laufzeit weitere Casts notwendig sind.

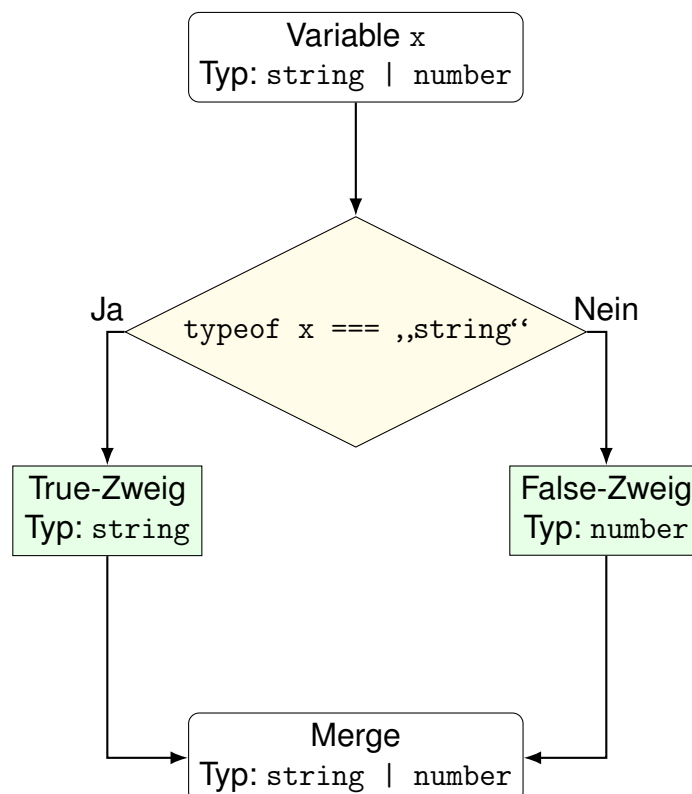


Abbildung 2: Control Flow Analysis: Der TypeScript-Compiler verfeinert (narrowt) den Typ basierend auf logischen Bedingungen.

2.1.3 Type Narrowing und Discriminated Unions

Eine zentrale Stärke von TypeScript ist die Fähigkeit, Typen innerhalb von Kontrollstrukturen zu verfeinern. Dieses als *Narrowing* bezeichnete Verfahren ermöglicht es dem Compiler, den Typ einer Variable in einem bestimmten Codeblock präzise einzugrenzen.

Besonders relevant ist dies bei der Verwendung von *Union Types* in Kombination mit Klassen. Der Operator `instanceof` fungiert in TypeScript als sogenannter *Type Guard*. Listing 1 illustriert dieses Prinzip anhand einer rekursiven Ausdrucksstruktur.

```
1 class Value {
2     constructor(public val: number) {}
3 }
4
5 class Add {
6     constructor(public left: Expr, public right: Expr) {}
7 }
8
9 type Expr = Value | Add;
10
11 function eval(e: Expr): number {
12     if (e instanceof Value) {
13         return e.val;
14     }
15
16     return eval(e.left) + eval(e.right);
17 }
```

Listing 1: Type Narrowing mittels `instanceof` und Control Flow Analysis

Der Code demonstriert das Zusammenspiel mehrerer TypeScript-Konzepte. Die Klassen `Value` und `Add` definieren die Struktur der Daten, wobei `Value` einen primitiven Wert hält und `Add` rekursiv auf zwei weitere Ausdrücke verweist. Der Typ `Expr` wird als Vereinigung (*Union Type*) dieser beiden Klassen definiert, sodass eine Variable dieses Typs zur Laufzeit entweder eine Instanz von `Value` oder von `Add` sein kann.

Innerhalb der Funktion `eval` dient die Prüfung mittels `instanceof` als Type Guard. Der Compiler erkennt, dass im ersten `if`-Block die Variable `e` zwingend vom Typ `Value` ist, wodurch der Zugriff auf die Eigenschaft `val` statisch erlaubt wird. Im weiteren Verlauf greift die *Control Flow Analysis*: Da der erste Pfad die Funktion durch ein `return` verlässt, schlussfolgert der Compiler für den verbleibenden Code, dass `e` nur noch vom Typ `Add` sein kann. Ein expliziter `else`-Block oder eine

erneute Prüfung sind somit überflüssig; der Zugriff auf `left` und `right` ist typsicher gewährleistet.

Dies unterscheidet sich fundamental von Python, wo ein fehlendes Attribut erst zur Laufzeit einen Fehler verursachen würde. In TypeScript garantiert das Typsystem die strukturelle Integrität der Fallunterscheidung bereits vor der Ausführung.

2.1.4 Strukturelle Typisierung (Structural Typing)

Ergänzend zum Narrowing ist die *strukturelle Typisierung* von TypeScript hervorzuheben. Im Gegensatz zu nominalen Systemen (wie Java oder C#) entscheidet nicht der Name einer Klasse oder eine explizite `implements`-Klausel über die Kompatibilität, sondern ausschließlich die Form (*Shape*) des Objekts.

Seien A und B zwei Objekttypen. Der Typ A gilt als kompatibel zu (oder Subtyp von) B , formal notiert als $A \subseteq B$, wenn folgende Bedingung erfüllt ist:

$$A \subseteq B \iff \forall p \in \text{properties}(B) : p \in \text{properties}(A) \wedge \text{type}_A(p) \subseteq \text{type}_B(p)$$

Hierbei bezeichnet:

- A : Den Typ des zuzuweisenden Objekts (Kandidat).
- B : Den erwarteten Zieltyp (Interface oder Klasse).
- p : Eine Eigenschaft (Property) oder Methode des Typs.

In Worten: Damit A anstelle von B verwendet werden kann, muss A mindestens alle Eigenschaften besitzen, die in B definiert sind, und deren Typen müssen wiederum kompatibel sein. Zusätzliche Eigenschaften in A stören die Kompatibilität nicht.

Listing 2 verdeutlicht dieses Prinzip. Ein Interface definiert lediglich eine Anforderung an die Struktur. Jedes Objekt, das diese Anforderung erfüllt, wird vom Typsystem akzeptiert.

```

1 interface Vector2D {
2     x: number;
3     y: number;
4 }
5
6 function calculateLength(v: Vector2D): number {
7     return Math.sqrt(v.x * v.x + v.y * v.y);
8 }
9
10 class Particle {
11     constructor(
12         public x: number,
13         public y: number,
14         public velocity: number
15     ) {}
16 }
17
18 const p = new Particle(3, 4, 10);
19 const len = calculateLength(p);
20 const len2 = calculateLength({ x: 5, y: 12 });

```

Listing 2: Implizite Kompatibilität ohne explizite Implementierung

Im gezeigten Beispiel wird die Klasse `Particle` (A) von der Funktion `calculateLength` akzeptiert, die `Vector2D` (B) erwartet. Da eine Instanz von `Particle` die geforderten Eigenschaften `x` und `y` besitzt (und mit `velocity` sogar zusätzliche Eigenschaften), gilt die Bedingung $A \subseteq B$ als erfüllt.

Dies erleichtert die Portierung von Python-Code erheblich, da Python's *Duck Typing*-Philosophie („Wenn es wie eine Ente läuft, ist es eine Ente“) direkt auf TypeScript's Interfaces abgebildet werden kann. Ein Algorithmus, der beispielsweise erwartet, dass ein Objekt über eine `hashCode`-Methode verfügt, kann in TypeScript durch ein entsprechendes Interface typisiert werden, ohne dass die verwendeten Klassen in eine starre Vererbungshierarchie gezwungen werden müssen.

Obwohl die explizite Verwendung von `implements` für die Typkompatibilität technisch nicht erforderlich ist, wird sie in der Praxis dennoch empfohlen. Sie verlagert die Fehlererkennung von der Aufrufstelle (*Call Site*) zur Definitionsstelle (*Definition Site*). Würde in der Klasse `Particle` beispielsweise die Eigenschaft `x` versehentlich fehlen, würde der Compiler dies ohne `implements` erst beim Aufruf von `calculateLength(p)` bemängeln. Mit einer expliziten Klausel (`class Particle implements Vector2D`) wird der Fehler direkt an der Klasse markiert, was die Intention des Entwicklers dokumentiert und die Wartbarkeit erhöht.

2.2 Algorithmik von Mengen und Abbildungen

Die effiziente Implementierung von Automatenalgorithmen (z. B. Potenzmengenkonstruktion, Minimierung) erfordert Datenstrukturen, die Mengen von Mengen verwalten können. Da Standardbibliotheken in JavaScript/TypeScript hierfür keine native Unterstützung bieten, ist ein tiefes Verständnis der zugrundeliegenden Hashing- und Speicherverfahren notwendig.

2.2.1 Identität und Gleichheit (Value Semantics)

In der Mengenlehre gilt das Extensionalitätsaxiom: Zwei Mengen sind genau dann gleich, wenn sie dieselben Elemente enthalten ($A = B \iff \forall x : (x \in A \iff x \in B)$). Diese Definition entspricht der *Wertgleichheit* (Value Equality). Programmiersprachen unterscheiden jedoch technisch zwischen:

1. **Referenzgleichheit (Identity):** Zwei Variablen zeigen auf dieselbe Speicheradresse. Dies ist das Standardverhalten von Set und Map in JavaScript für Objekte.
2. **Strukturelle Gleichheit (Deep Equality):** Zwei Objekte sind äquivalent, wenn ihre rekursive Struktur und ihre primitiven Werte übereinstimmen.

Für die Portierung der Python-Logik, in der Tupel und 'frozensets' als Schlüssel in Dictionaries dienen, ist die Implementierung struktureller Gleichheit zwingend erforderlich. Dies impliziert, dass auch der Hash-Wert eines Objekts nicht aus seiner Speicheradresse, sondern deterministisch aus seinem Inhalt abgeleitet werden muss.

2.2.2 Hash-Funktionen und Verteilung

Eine Hash-Funktion $h : D \rightarrow I$ bildet Daten beliebiger Größe auf einen Indexbereich ab. Für performante Hash-Tabellen sind drei Eigenschaften entscheidend:

- **Determinismus:** Gleicher Input muss stets den gleichen Hash-Wert liefern. Dies ist bei rekursiven Strukturen nichttrivial, da die Iterationsreihenfolge von Elementen in Standard-Sets undefiniert ist. Eine kanonische Sortierung (z. B. nach Hash-Werten der Kinder) ist daher vor dem Hashing notwendig.
- **Uniformität:** Die Hash-Werte sollten gleichmäßig über den Wertebereich verteilt sein, um Kollisionen zu minimieren. Moderne Verfahren nutzen hierzu Bit-Operationen (XOR, Shift) und Multiplikationen mit großen Primzahlen, um einen „Avalanche Effect“ zu erzielen, bei dem die Änderung eines Eingabebits das Umkippen vieler Ausgabebits bewirkt [cormen2022introduction].

- **Performance:** Da Hash-Berechnungen in inneren Schleifen von Automatenalgorithmen stattfinden, müssen Allokationen (z. B. Strings-Konkatenation) vermieden werden.

2.2.3 Kollisionsauflösung: Open Addressing und SoA

Treffen zwei unterschiedliche Schlüssel auf denselben Hash-Index (Kollision), muss eine Ausweichstrategie gewählt werden. Während viele Standardimplementierungen (wie Java `HashMap`) auf *Chaining* (verkettete Listen pro Bucket) setzen, bietet das *Open Addressing* (offene Adressierung) signifikante Vorteile für die Cache-Lokalität.

Beim Open Addressing werden alle Elemente direkt im Array der Tabelle gespeichert. Bei einer Kollision an Index i wird eine Sondierungsfolge (Probing Sequence) durchlaufen, typischerweise *Linear Probing*:

$$\text{index}(k, j) = (h(k) + j) \pmod{m}$$

wobei j der Sondierungsschritt ist.

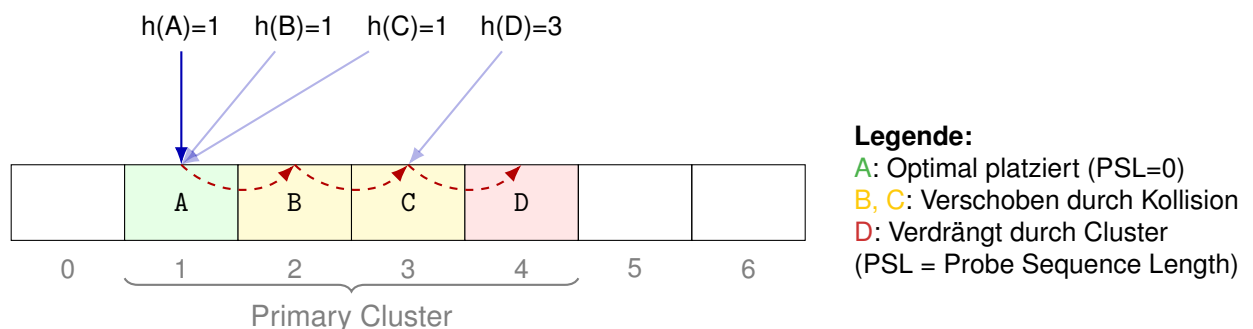


Abbildung 3: Visualisierung von Linear Probing und Clusterbildung. Elemente B und C kollidieren mit A. Element D (Hash 3) trifft auf das belegte Feld 3 und muss auf 4 ausweichen, obwohl es gar nicht Hash 1 hat.

Um die Performance weiter zu steigern, kann das Speicherlayout nach dem Prinzip **Structure of Arrays (SoA)** optimiert werden. Anstatt ein Array von Objekten (`{key, value, hash}`) zu speichern, werden drei separate, flache Arrays (`keys[]`, `values[]`, `hashes[]`) verwaltet. Dies erhöht die räumliche Lokalität (Spatial Locality) im CPU-Cache, da beim Durchsuchen der Tabelle nur die relevanten Datenströme geladen werden müssen.

2.2.4 Löschrategien: Backshift Deletion

Eine Schwäche des Linear Probing ist das Löschen von Elementen. Wird ein Element einfach entfernt, entsteht eine Lücke, die die Sondierungskette für nachfol-

gende Elemente unterbricht. Klassische Implementierungen nutzen daher „Tombstones“ (Grabsteine) – spezielle Marker für gelöschte Felder. Diese haben jedoch den Nachteil, dass sie die Tabelle „verschmutzen“ und die Suchzeit bei vielen Einfüge- und Löschoptionen degradieren.

Eine effizientere, wenn auch implementierungstechnisch aufwendigere Methode ist das **Backshift Deletion** (auch bekannt als *Backward Shift Deletion*). Hierbei wird beim Löschen an Index i geprüft, ob das nachfolgende Element an $i + 1$ (oder weiter) nur aufgrund einer Kollision dort liegt und eigentlich an eine Position $\leq i$ gehört. Ist dies der Fall, wird das Element nach i verschoben („zurückgerückt“), und die Prüfung setzt sich rekursiv fort. Dieses Verfahren hält die Tabelle kompakt, eliminiert die Notwendigkeit von Tombstones vollständig und garantiert, dass die Suchpfade minimal bleiben. Es ist damit besonders geeignet für langlebige Datenstrukturen in einer Interpreter-Laufzeitumgebung.

2.3 Termersetzung und Pattern Matching

Für die Optimierung regulärer Ausdrücke und die symbolische Manipulation von Syntaxbäumen greift die Arbeit auf Konzepte der Termersetzung (Term Rewriting Systems, TRS) zurück. Ein TRS ist ein Berechnungsparadigma, bei dem Ausdrücke durch die gerichtete Anwendung von Regeln schrittweise transformiert werden, bis eine Normalform erreicht ist [baader1998term].

2.3.1 Termersetzungssysteme (TRS)

Formal ist ein TRS definiert als eine Menge von Regeln $l \rightarrow r$, wobei l und r Terme sind. Eine Regel ist anwendbar auf einen Term t , wenn l durch Substitution von Variablen auf einen Unterterm von t abgebildet werden kann. Im Kontext dieser Arbeit wird ein TRS verwendet, um die Axiome der Kleene-Algebra (Reguläre Algebra) operativ umzusetzen. Typische Regeln sind:

$$R + \emptyset \rightarrow R \quad (\text{Neutrales Element der Vereinigung})$$

$$R \cdot \varepsilon \rightarrow R \quad (\text{Neutrales Element der Konkatenation})$$

$$R + R \rightarrow R \quad (\text{Idempotenz})$$

Das Ziel ist die Reduktion komplexer, maschinengenerierter Ausdrücke (wie sie bei der Konvertierung von Automaten entstehen) in eine menschlich lesbare Minimalform.

2.3.2 Pattern Matching und Unifikation

Die technische Umsetzung eines TRS erfordert einen *Pattern Matching*-Algorithmus. Gegeben sei ein Muster p (das Variablen enthalten kann, z. B. $R + \emptyset$) und ein konkreter Term t (z. B. $(a \cdot b) + \emptyset$). Das Matching-Problem besteht darin, eine Substitution σ zu finden, sodass gilt:

$$\sigma(p) \equiv t$$

Dabei bildet σ jede Variable in p auf einen Unterbaum von t ab. Im Beispiel wäre $\sigma(R) = (a \cdot b)$. Die Implementierung in TypeScript (siehe `RegExp-Tutorial.ipynb`) realisiert dies durch eine rekursive `match`-Funktion, die eine *Bindings-Map* aufbaut. Ein besonderer Aspekt ist hierbei die Prüfung auf Konsistenz bei nicht-linearen Mustern (z. B. $R + R \rightarrow R$): Wenn eine Variable mehrfach im Muster vorkommt, müssen alle gematchten Unterbäume strukturell identisch sein (Deep Equality), was die in Abschnitt 2.2 eingeführte Wertsemantik voraussetzt.

2.3.3 Fixpunkt-Iteration

Da die Anwendung einer Regel neue Reduktionsmöglichkeiten schaffen kann (z. B. führt $0^* \rightarrow \varepsilon$ dazu, dass ein übergeordnetes $R \cdot \varepsilon$ zu R wird), muss der Rewriting-Prozess iterativ erfolgen. Ein Term t befindet sich in Normalform, wenn keine Regel mehr anwendbar ist. Der Algorithmus wendet die Regeln solange an, bis ein Fixpunkt erreicht ist:

$$t_{n+1} = \text{rewrite}(t_n) \quad \text{mit} \quad t_{n+1} \equiv t_n$$

Dieses Fixpunkt-Prinzip findet sich in der Arbeit nicht nur bei regulären Ausdrücken wieder, sondern ist auch die mathematische Basis der ε -Closure-Berechnung bei endlichen Automaten (siehe `01-NFA-2-DFA.ipynb`).

2.4 Formale Sprachen und Syntaxanalyse

Die Überführung von Quelltext in eine ausführbare Struktur erfolgt in mehreren Stufen der Abstraktion. Die Arbeit nutzt hierfür den modernen Parser-Generator *Lezer*, dessen theoretische Einordnung im Folgenden kurz skizziert wird.

2.4.1 LR-Parsing und der Concrete Syntax Tree (CST)

Die Syntaxanalyse basiert auf einer kontextfreien Grammatik (CFG), die häufig in der Erweiterten Backus-Naur-Form (EBNF) notiert wird. *Lezer* generiert aus dieser Grammatik einen inkrementellen LR-Parser (Left-to-right, Rightmost derivation).

Das Ergebnis des Parsvorgangs ist ein *Concrete Syntax Tree* (CST). Der CST ist eine flache, vollständige Repräsentation des Quelltextes, die alle Terminalsymbole, inklusive Klammern, Kommata und Kommentare, enthält.

Für die algorithmische Weiterverarbeitung (Evaluierung, Kompilierung) ist dieser Detailgrad hinderlich, da er syntaktisches Rauschen enthält.

2.4.2 Transformation zum Abstrakten Syntaxbaum (AST)

Ein zentraler Schritt der Implementierung ist die Überführung des CST in einen *Abstract Syntax Tree* (AST). Der AST abstrahiert von der konkreten Syntax und repräsentiert nur die logische Struktur des Programms. Die Transformation $T : \text{CST} \rightarrow \text{AST}$ erfolgt in der Arbeit nicht durch strikte 1:1-Abbildung jeder Produktionsregel, sondern durch heuristische Verfahren (siehe `cst2ast`), um eine idiomatische Baumstruktur zu erzeugen:

- **Pivot Promotion (Infix-Operatoren):** In der konkreten Syntax stehen Operatoren oft als Geschwisterknoten (z. B. `[Expr, +, Expr]`). Im AST wird der Operator zur Wurzel des Teilbaums erhoben (*Pivot*), während die Operanden zu Kindern werden. Dies stellt die Operator-Präzedenz strukturell dar.
- **Listen-Faltung (Cons-Lists):** Während Lezer Listen flach als Sequenz von Kindern liefert, modelliert die AST-Definition Listen als rekursive Tupel der Form `[',', Head, Tail]`. Diese *Cons-Cell*-Struktur, bekannt aus LISP und funktionalen Sprachen, erlaubt eine einfache rekursive Verarbeitung im Interpreter ohne Index-Management.
- **Implizite Aufrufe:** Der Transformator unterscheidet kontextbasiert zwischen Variablenzugriffen und Funktionsaufrufen, indem er auf die Existenz von Klammer-Token im CST prüft, selbst wenn die Grammatik diese Unterscheidung nicht explizit benennt.

2.5 Laufzeitumgebungen und WebAssembly

Ein technisches Nebenziel der Arbeit ist die Plattformunabhängigkeit der Visualisierungskomponenten. Traditionell erfordert die Darstellung von Automaten-Graphen die Installation der Graphviz-Binaries auf dem Host-System. WebAssembly (WASM) ist ein binäres Instruktionsformat für eine stapelbasierte virtuelle Maschine, das als portables Zielformat für die Kompilierung von Hochsprachen wie C++ dient [[haas2017bringing](#)]. Durch die Nutzung von `@viz-js/viz` wird der ursprüngliche C-Code von Graphviz zu WASM kompiliert. Dies ermöglicht die Ausführung der komplexen Layout-Algorithmen direkt innerhalb der JavaScript-Laufzeitumgebung (Browser oder Node.js), isoliert vom Betriebssystem und ohne

native Installationen. Dies erfüllt die Anforderung der Interoperabilität (vgl. Abschnitt 1.2).

3 Lezer als Parser-Generator

Dieses Kapitel behandelt die theoretischen Grundlagen und die praktische Einbindung von *Lezer* als Parser-Generator im Kontext der Übersetzung von Python-Programmen nach TypeScript. Während in Kapitel 1 die typentheoretischen Grundlagen von TypeScript analysiert wurden und Kapitel 2 geeignete Datenstrukturen diskutierte, wird nun die syntaktische Ebene betrachtet: die formale Beschreibung, Analyse und Transformation von Programmen mittels Grammatiken und Parsern.

3.1 Einordnung in die Theorie formaler Sprachen

Die syntaktische Analyse basiert auf dem Modell kontextfreier Grammatiken (Chomsky-Typ 2). Eine kontextfreie Grammatik ist ein vierfach Tupel $G = (V, \Sigma, P, S)$ bestehend aus einer Menge von syntaktische Variablen oder Nichtterminalen V , Terminalen oder Token Σ , Grammatikregeln P und einem Startsymbol S [4, S.171ff].

Die zentrale Aufgabe eines Parsers besteht darin, für eine Eingabesequenz ein Ableitungs- bzw. Parsebaum-Konstrukt zu bestimmen. Ein Parsebaum repräsentiert die hierarchische Struktur der Ableitung und ist Grundlage für weitere semantische Transformationen [5, S.61f].

Für die vorliegende Projektarbeit ist insbesondere die Unterscheidung zwischen Top-Down- und Bottom-Up-Verfahren relevant. LR-Verfahren (Left-to-right, Rightmost derivation in reverse) gehören zur Klasse der deterministischen Bottom-Up-Parser und besitzen für kontextfreie Grammatiken eine hohe Ausdrucksstärke [5, S.82f].

3.2 Lezer: Konzept und Architektur

Lezer ist ein moderner Parser-Generator, der im Umfeld des CodeMirror-Editors entwickelt wurde. Er erzeugt effiziente LR-basierte Parser in JavaScript bzw. TypeScript [6].

Im Unterschied zu klassischen Parsergeneratoren wie Yacc [7, S.287f] verfolgt Lezer einen stark inkrementellen Ansatz. Das bedeutet, dass bei Änderungen am Quelltext nicht der gesamte Parsebaum neu konstruiert wird, sondern nur die betroffenen Teilbereiche aktualisiert werden.

Formal entspricht die von Lezer verarbeitete Grammatik einer kontextfreien Grammatik mit expliziten Präzedenz- und Assoziativitätsangaben. Diese Technik dient der Auflösung klassischer Mehrdeutigkeiten, etwa bei arithmetischen Ausdrücken [4, S.210].

3.3 Grammatikdefinition in Lezer

Eine Lezer-Grammatik wird deklarativ beschrieben. Beispielhaft sei eine stark vereinfachte Ausdrucksgrammatik angegeben:

```
1  const grammarDefinition = `
2      @top Program { Statement+ }
3
4      @tokens {
5          "+" "-" "*" "/" ":@"
6          Number      { $[0-9]+ ("." $[0-9]*)? }
7          Identifier  { $[a-zA-Z] $[a-zA-Z0-9_]* }
8          space       { $[ \t\r\n]+ }
9      }
10
11     Statement {
12         Identifier ":" Expr ";" |
13         Expr ";"
14     }
15
16     Expr {
17         Expr "+" Product |
18         Expr "-" Product |
19         Product
20     }
21
22     Product {
23         Product "*" Factor |
24         Product "/" Factor |
25         Factor
26     }
27
28     Factor {
29         "(" Expr ")" |
30         Number      |
31         Identifier
32     }
33
34     @skip { space }
35 `;
```

Formal entspricht dies der kontextfreien Grammatik:

$$\begin{aligned}
stmt &\rightarrow IDENTIFIER' :=' expr ';' \\
&| \quad expr ';' \\
expr &\rightarrow expr' +' product \\
&| \quad expr' -' product \\
&| \quad product \\
product &\rightarrow product' '*' factor \\
&| \quad product' '/' factor \\
&| \quad factor \\
factor &\rightarrow '(' expr ')' \\
&| \quad NUMBER \\
&| \quad IDENTIFIER
\end{aligned}$$

Die linke Rekursion wird von LR-basierten Verfahren direkt unterstützt, im Gegensatz zu LL-Verfahren, bei denen sie zuvor eliminiert werden müsste [7, S.212ff].

3.4 Parsebäume und abstrakte Syntaxbäume

Ein wesentlicher Schritt bei der Übersetzung von Python nach TypeScript besteht in der Konstruktion eines abstrakten Syntaxbaums (AST). Der konkrete Parsebaum enthält syntaktische Details, während der AST auf die semantisch relevanten Knoten reduziert wird [7, S.358f.].

Lezer erzeugt intern eine kompakte Baumrepräsentation, die inkrementell aktualisierbar ist. Für Transformationszwecke wird dieser Baum traversiert und in eine eigene AST-Struktur überführt, welche anschließend zur Codegenerierung genutzt wird.

Die formale Korrektheit dieses Vorgehens lässt sich als Homomorphismus zwischen zwei Baumstrukturen interpretieren:

$$h : T_{\text{Python}} \rightarrow T_{\text{TypeScript}}$$

wobei T_{Python} der durch die Python-Grammatik erzeugte Baum ist und $T_{\text{TypeScript}}$ die Zielstruktur repräsentiert.

3.5 Inkrementelles Parsing und Komplexität

Klassische LR-Parser besitzen im deterministischen Fall lineare Laufzeit in der Länge der Eingabe [4, S.73]. Lezer erweitert dieses Modell um inkrementelle Aktualisierung.

Sei n die Länge des Quelltexts und k die Länge einer lokalen Änderung, so wird im Idealfall nur ein Teilbaum der Größe $\mathcal{O}(k)$ neu berechnet. Dies ist insbesondere für editorintegrierte Anwendungen relevant.

3.6 Bedeutung für die Python–TypeScript-Übersetzung

Im Rahmen der Studienarbeit übernimmt Lezer folgende Aufgaben:

- Formale Spezifikation der Python-Teilgrammatik
- Konstruktion eines syntaktisch korrekten Parsebaums
- Bereitstellung einer strukturierten Baumrepräsentation für die Transformation

Die syntaktische Korrektheit ist Voraussetzung für die spätere Typtransformation und Codegenerierung. Fehlerhafte oder mehrdeutige Grammatiken würden direkt zu fehlerhaften Übersetzungen führen.

3.7 Zusammenfassung Kapitel 3

Lezer stellt einen modernen, LR-basierten Parser-Generator dar, der sich aufgrund seiner inkrementellen Architektur besonders für editornahe Anwendungen eignet. Formal basiert er auf kontextfreien Grammatiken und deterministischen Bottom-Up-Verfahren.

Für die vorliegende Studienarbeit bildet Lezer die Brücke zwischen der formalen Beschreibung der Python-Syntax und der strukturierten Transformation in typisierte TypeScript-Programme.

Im folgenden Kapitel wird die konkrete Modellierung der Python-Grammatik und deren Anpassung an die Anforderungen der Übersetzung detailliert analysiert.

4 Transformation und Codegenerierung

Nachdem in Kapitel 3 die syntaktische Analyse mittels *Lezer* behandelt wurde, steht nun die semantische Verarbeitung des erzeugten Syntaxbaums im Fokus. Ziel ist eine Abbildung eines Python-Programms bzw. der Python-Lehrbeispiele auf ein strukturell und typologisch äquivalentes TypeScript-Programm. Dabei umfasst “Transformation” sowohl (i) die Zuordnung/Explizierung von Typinformationen als auch (ii) die Anpassung an syntaktische und semantische Unterschiede zwischen beiden Sprachen, bevor abschließend (iii) TypeScript-Quelltext generiert wird.

4.1 Einordnung als syntaxgerichtete Übersetzung

Die Umsetzung folgt dem Prinzip der *syntaxgerichteten Übersetzung*: Grammatiksymbolen werden Attribute zugeordnet, deren Werte während der Traversierung berechnet werden [7, S.304ff]. In der Praxis werden diese Attribute im Projekt insbesondere genutzt, um (a) typologische Informationen zu propagieren (z. B. "Zahl", "Tokenliste", "Automatenzustand"), und (b) passende Zielkonstrukte in TypeScript zu wählen (z. B. Tupeltypen, Union-Typen oder projektinterne Container mit struktureller Gleichheit).

Eine syntaxgerichtete Definition kann als Tupel (G, A, R) modelliert werden, wobei G eine kontextfreie Grammatik bezeichnet, A eine Attributmenge, und R die Menge semantischer Regeln zur Attributberechnung.

4.2 Abbildung von Python- auf TypeScript-Typen (erweitert)

Ein wesentlicher Unterschied zwischen Python und TypeScript liegt in der Typisierung. Python ist dynamisch typisiert, während TypeScript eine statische Typprüfung durchführt [2, S.1ff]. Für die Portierung bedeutet dies: Datenformen, die in Python *implizit* zur Laufzeit entstehen, müssen in TypeScript *explizit* und konsistent modelliert werden.

TypeScript ist zudem *strukturell* typisiert: Typkompatibilität ergibt sich aus der Struktur, nicht aus nominalen Typnamen. Dadurch lassen sich viele Python-Datenmodelle direkt als Objekt-/Record-Strukturen oder als Union-Typen ausdrücken [8, S.44ff].

4.2.1 Grundabbildungen

Für Basis- und Standard-Container-Typen ergibt sich zunächst eine direkte Abbildung:

Python	TypeScript	Bemerkung
int, float	number	gemeinsame Zahlendomäne
bool	boolean	direkt
str	string	direkt
list[T]	T[] / Array<T>	sequenziell, indexierbar
tuple[T1,T2]	[T1,T2]	feste Länge/Positionen
X Y	X Y	Union-Typen (mit Narrowing)
Literal["A","B"]	"A" "B"	String-Literal-Union
None / optional	null / undefined	je nach Modellierung

Union-Typen werden im Projekt genutzt, um Python-typische Mehrdeutigkeiten (z. B. “Zahl *oder* AST-Knoten”) statisch auszudrücken. Durch Guards/Narrowing können gültige Fälle schrittweise eingegrenzt werden [8, S.73ff].

4.2.2 Unions, Literale und Generics (Beispiele aus der Portierung)

Im Python-Notebook werden Unions (PEP 604), Literale und TypeVar genutzt; in TypeScript werden diese als Union-Typen, String-Literal-Unions und Generics umgesetzt:

Beispielcode in Python

```
1 Name      = str | dict[str, str]
2 Direction = Literal["North", "East", "South", "West"]
3
4 a = TypeVar("a")
5 b = TypeVar("b")
6 def swap(x: a, y: b) -> tuple[b, a]:
7     return (y, x)
```

Beispielcode in TypeScript

```
1 type Name = string | Record<string, string>;
2 type Direction = "North" | "East" | "South" | "West";
3
4 function swap<a, b>(x: a, y: b): [b, a] {
5     return [y, x];
6 }
```

4.2.3 Schlüsselstrukturen: tuple/frozenset als Keys

Ein zentraler Unterschied betrifft Python-typische Muster wie dict mit tuple-Schlüsseln oder frozenset (hashbar, unveränderlich). In JavaScript/TypeScript verwenden Map/Set bei Objekt-/Array-Keys standardmäßig *Referenzgleichheit*; in Python basieren tuple/frozenset hingegen auf *Wertgleichheit* (Inhalt). Damit Algorithmen (z. B. Determinisierung, Minimierung, Äquivalenzprüfung) unverändert funktionieren, werden im Projekt strukturell vergleichbare Container eingesetzt: Tuple<...> (wertbasierte Tupel) sowie RecursiveSet<T> und RecursiveMap<K,V> (Containers mit struktureller Gleichheit und Hashbarkeit).

4.2.4 Automaten: set/frozenset, Transitionen und Zustandsmengen

Python:

Zustandsmengen werden für Immutability und Hashbarkeit als frozenset modelliert; Transitionen sind dict mit tuple-Schlüsseln:

Beispielcode in Python

```
1 State      = TypeVar("State", str, int)
2 Char       = str
3 TransRel   = dict[tuple[State, Char], set[State]]
4 TransRelDet = dict[tuple[frozenset[State], Char], frozenset[State]]
5
6 NFA = tuple[set[State], set[Char], TransRel, State, set[State]]
7 DFA = tuple[
8     set[frozenset[State]], set[Char], TransRelDet,
9     frozenset[State], set[frozenset[State]]
10 ]
```

TypeScript:

Für strukturelle Gleichheit (gleicher Inhalt statt gleicher Referenz) werden `RecursiveSet` und `RecursiveMap` verwendet; komplexe Schlüssel werden über `Tuple<...>` repräsentiert:

Beispielcode in TypeScript

```
1 type State = string | number;
2 type Char = string;
3
4 type DFAState = RecursiveSet<State>;
5
6 type TransRel =
7     RecursiveMap<Tuple<[State, Char]>, RecursiveSet<State>>;
8
9 type TransRelDet =
10     RecursiveMap<Tuple<[DFAState, Char]>, DFAState>;
11
12 interface NFA {
13     Q: RecursiveSet<State>;
14     Sigma: RecursiveSet<Char>;
15     delta: TransRel;
16     q0: State;
17     A: RecursiveSet<State>;
18 }
19
20 interface DFA {
21     Q: RecursiveSet<DFAState>;
22     Sigma: RecursiveSet<Char>;
23     delta: TransRelDet;
24     q0: DFAState;
25     A: RecursiveSet<DFAState>;
26 }
```

Damit bleibt die in Python “natürliche” Nutzung von Mengen/Tupeln als Schlüssel (z. B. im Determinisierungsalgorithmus) in TypeScript semantisch erhalten,

ohne auf any oder manuelle Serialisierung ausweichen zu müssen.

4.2.5 Rekursive Strukturen: ParseTree, RegExp und AST

Viele Lehrbeispiele modellieren Bäume rekursiv. Python nutzt hierfür häufig verschachtelte Tupel und Unions; TypeScript bildet dies entweder als rekursive Union-Typen oder als explizite Tuple-Formen ab.

ParseTree (RegExp):

Beispielcode in Python

```
1 ParseTree = tuple[str, list["ParseTree"]]
2 ParseTreeList = list[ParseTree]
```

Beispielcode in TypeScript

```
1 type ParseTree = Tuple<string, ParseTreeList>;
2 type ParseTreeList = ParseTree[];
```

AST (Ausdrucksbäume):

In Python wird der AST häufig als “Atom oder verschachteltes Tupel” modelliert; eine Umgebung als dict:

Beispielcode in Python

```
1 NestedTuple = TypeVar('NestedTuple')
2 NestedTuple = int | str | tuple[NestedTuple, ...]
3 Number = int | float
```

In TypeScript wird dies in ein rekursives AST-Union überführt, plus explizite Environment-Struktur:

Beispielcode in TypeScript

```
1 type Operator = string;
2 type AST = string | number |
3     [Operator, AST] |
4     [Operator, AST, AST] |
5     [Operator, AST, AST, AST] |
6     [Operator, AST, AST, AST, AST];
```

Die Fallunterscheidung erfolgt in TypeScript entsprechend über die Tupellänge und ggf. über Guards (z. B. `Array.isArray(x)` und `x.length`), wodurch die Auswertung/Transformation analog zum Python-Code umgesetzt werden kann.

4.2.6 Parser-Rückgabewerte: `tuple` $\rightarrow$ TS-Tupeltyp

Für rekursive Descent-Parser liefern Python-Funktionen häufig ein Paar aus “Wert” und “Resttokenliste” zurück. In TypeScript wird dies als Tupeltyp modelliert:

Beispielcode in Python

```
1 def parseExpr(TL: list[str]) -> tuple[float, list[str]]:
2     ...
3 def parseFactor(TL: list[str]) -> tuple[float, list[str]]:
4     ...
```

Beispielcode in TypeScript

```
1 type TokenList = string[];
2 type ParseResult = [number, TokenList];
3 type Parser = (TL: TokenList) => ParseResult;
4
5 function parseExpr(TL: TokenList): ParseResult { /* ... */ }
6 function parseFactor(TL: TokenList): ParseResult { /* ... */ }
```

Zwischenfazit.

Die Portierung nutzt Standardabbildungen (z. B. `list[T]  $\rightarrow$  T[]`) dort, wo TypeScript dieselbe Semantik bietet. Für Python-typische Schlüsselstrukturen (`tuple/frozenset` als Keys) sind jedoch eigene, strukturell vergleichbare Container notwendig, um die Algorithmen (Determine/Minimize/Equivalence) ohne Semantikverlust in TypeScript ausdrücken zu können.

4.3 Transformation des abstrakten Syntaxbaums

Die eigentliche Transformation erfolgt durch rekursive Traversierung des Syntaxbaums. Dabei wird jeder Python-Knoten (bzw. jede Python-Konstruktion) auf einen korrespondierenden TypeScript-Knoten abgebildet. Formal sei $t \in T_{\text{Python}}$ ein Syntaxbaum, dann ist die Transformation eine Funktion

$$f(t) = t'$$

mit $t' \in T_{\text{TypeScript}}$.

Die Transformation ist *strukturerhaltend*: Die Hierarchie der Ausdrücke bleibt erhalten, während (i) Typannotationen ergänzt, (ii) konkrete Syntaxformen angepasst und (iii) notwendige Hilfskonstrukte (z. B. Guards) eingeführt werden.

4.4 Behandlung semantischer Unterschiede

Python erlaubt z. B. implizite Wahrheitswerte für beliebige Objekte, während TypeScript eine strengere Typdisziplin verlangt. Ein Python-Konstrukt wie

```

1 if x:
2     print(x)

```

wird daher in TypeScript explizit konvertiert bzw. durch Guards abgesichert:

```

1 if (Boolean(x)) {
2     console.log(x);
3 }

```

Je nach Datentyp kann statt `Boolean(x)` auch ein präziser Guard sinnvoll sein (z. B. `x !== undefined && x !== null` oder `typeof x === "number"`). Solche Anpassungen sind Teil der semantischen Phase der Übersetzung, da sie nicht rein syntaktisch entschieden werden können.

4.5 Erzeugung von TypeScript-Code

Die Codegenerierung erfolgt nach der Baumtransformation durch Serialisierung des Ziel-IR (bzw. Ziel-AST) in TypeScript-Quelltext. Dabei werden insbesondere

- Typannotationen eingefügt (z. B. `: number`, `: ParseResult`),
- Blockstrukturen an TypeScript angepasst,
- optionale syntaktische Normalisierungen vorgenommen (z. B. Semikolons, Klammerung).

Formal entspricht die Generierung einer linearen Projektion des Baums:

$$g : T_{\text{TypeScript}} \rightarrow \Sigma^*$$

wobei Σ^* die Menge aller möglichen Quelltextstrings bezeichnet.

4.6 Korrektheit der Transformation

Die Übersetzung ist korrekt, wenn für jedes zulässige Python-Programm P gilt:

$$\llbracket P \rrbracket_{\text{Python}} = \llbracket f(P) \rrbracket_{\text{TypeScript}}$$

unter Berücksichtigung der eingeführten Typrestriktionen und ggf. notwendiger Guards. Dies entspricht dem Begriff der *semantischen Äquivalenz* [7, S.15ff].

In der praktischen Umsetzung wird diese Forderung typischerweise durch eine Kombination aus Testfällen (z. B. Referenz-Ausgaben) und Invarianten abgesichert

(z. B. “Übergangsfunktion bleibt total auf dem Alphabet”, “AST-Knotenformen sind vollständig abgedeckt”).

4.7 Zusammenfassung Kapitel 4

Kapitel 4 hat die Transformation von Python nach TypeScript als semantische Phase der Übersetzung modelliert: auf Basis syntaxgerichteter Regeln werden Typinformationen expliziert, Python-Datenmodelle in statisch typisierte TypeScript-Strukturen überführt und anschließend TypeScript-Quelltext generiert. Ein Schwerpunkt liegt dabei auf der typ- und strukturgetreuen Abbildung von Python-typischen Schlüsselstrukturen (Tupel/Mengen als Keys), um die Semantik der Lehalgorithmen in TypeScript ohne Semantikverlust zu erhalten.

5 Validierung, Tests und Grenzen der Portierung

Nachdem in Kapitel 4 die Transformation und Codegenerierung beschrieben wurde, behandelt dieses Kapitel die praktische Absicherung der Übersetzung: Wie wird überprüft, dass die TypeScript-Implementierungen (i) dieselbe Semantik wie die Python-Versionen besitzen und (ii) die neu eingeführten Typinvarianten tatsächlich eingehalten werden. Ergänzend werden typische Grenzen diskutiert, die sich aus Sprach- und Laufzeitunterschieden ergeben.

5.1 Statische Absicherung durch den TypeScript-Compiler

Ein zentraler Qualitätsgewinn der Portierung ist die frühzeitige Fehlererkennung durch den TypeScript-Compiler. Durch explizite Typen (insbesondere Union-Typen, Tupel-Varianten und generische Container) werden viele Fehlerklassen, die in Python erst zur Laufzeit sichtbar würden, bereits beim Übersetzen ausgeschlossen. In der Praxis betrifft dies vor allem:

- **Arity-/Formfehler in AST-ähnlichen Strukturen:** Operator-Knoten mit falscher Operandenanzahl werden durch Tupeltypen verhindert.
- **Inkonsistente Rückgabewerte:** Parser-Funktionen, die in Python Paare (Wert, Restliste) liefern, werden als Tupeltyp `[Value, TokenList]` modelliert; fehlende oder vertauschte Komponenten sind dadurch statisch auffindbar.
- **Fehlende Null-/Optional-Behandlung:** Optionale Werte werden explizit als `T | undefined` bzw. `T | null` modelliert, sodass Zugriff ohne Guard nicht möglich ist. [8, S.73ff]

Damit fungiert der TypeScript-Compiler als formales Prüfwerkzeug, das strukturelle Konsistenz über das gesamte Projekt hinweg erzwingt, ohne dass Laufzeitprüfungen notwendig werden. [8, S.44ff]

5.2 Dynamische Validierung durch Tests

Die in Kapitel 4 formulierte semantische Äquivalenz

$$\llbracket P \rrbracket_{\text{Python}} = \llbracket f(P) \rrbracket_{\text{TypeScript}}$$

ist im Projektkontext nicht vollständig formal bewiesen, sondern wird durch Tests und Vergleichsausführungen empirisch abgesichert. [7, S.15ff] Dabei werden drei Testebenen unterschieden:

5.2.1 Unit-Tests für lokale Funktionen

Für einzelne Komponenten (z. B. Tokenisierung, Hilfsfunktionen auf Mengen/Maps, einzelne Transformationsschritte) werden Eingabe-Ausgabe-Tests eingesetzt. Ziel ist, Grenzfälle (leere Eingaben, Sonderzeichen, ungewöhnliche Tokenfolgen) explizit abzudecken.

5.2.2 Integrations-Tests entlang der Pipeline

Integrations-Tests prüfen ganze Verarbeitungsketten, z. B.:

Source → Scanner → Parser → Transformation → Codegen

Hier steht weniger die isolierte Funktion, sondern die *Kompatibilität der Schnittstellen* im Vordergrund. Gerade die Portierung von “locker” typisierten Python-Schnittstellen auf strikt typisierte TypeScript-Schnittstellen deckt an dieser Stelle häufig Integrationsprobleme auf (z. B. falsche Tokenformen oder uneinheitliche AST-Knotenrepräsentationen).

5.2.3 Differentialtests: Python vs. TypeScript

Wo möglich, wird die TypeScript-Ausgabe direkt gegen das Verhalten der Python-Referenz implementiert (“gleiches Input-Set, gleiche Output-Relation”). Besonders geeignet sind hierfür deterministische Funktionen (z. B. Determinisierung/Minimierung von Automaten, Regex-ParseTree-Erzeugung, Parser-Auswertung arithmetischer Ausdrücke). Diese Vorgehensweise nähert die in Kapitel 4 genannte Korrektheitsforderung praktisch an, ohne vollständige Formalisierung.

5.3 Typinvarianten als Test

Ein Teil der Validierung wird bereits durch das Typmodell selbst übernommen: Viele Zustände sind *nicht konstruierbar*, wenn sie inkonsistent wären (z. B. falsche Schlüsseltypen in Transitionstabellen, unzulässige AST-Knotenform). Der Compiler wird damit zu einem Hilfsmittel, das ganze Fehlerklassen ausschließt. Dieses Prinzip entspricht der Idee, dass gut-typisierte Terme nicht “schiefgehen” (“do not go wrong”), da bestimmte fehlerhafte Zustände syntaktisch/typologisch nicht erreichbar sind. [2, S.1ff]

5.4 Grenzen und typische Stolperstellen

Trotz der strukturerhaltenden Übersetzung ergeben sich Grenzen aus den Unterschieden zwischen Python und TypeScript:

- **Gleichheit und Hashing komplexer Schlüssel:** Python erlaubt `tuple/frozenset` als `dict`-Keys mit inhaltlicher Gleichheit. In TypeScript erfordert dies projektspezifische Container (`RecursiveSet/RecursiveMap` bzw. `Tuple-Wrapper`), da `Map/Set` referenzbasiert arbeiten.
- **Zahlenmodell:** Python unterscheidet `int` und `float`, während TypeScript beide in `number` abbildet. Dadurch können sehr große Integer oder exakte Ganzzahlarithmetik nicht 1:1 garantiert werden.
- **Truthiness und implizite Konversionen:** Python erlaubt Bedingungen über beliebige Objekte; TypeScript verlangt typisch explizitere Konversionen bzw. Guards, um Narrowing zuverlässig zu erhalten. [8, S.73ff]
- **Lesbarkeit:** Im gegensatz zu Python lässt sich die Grammatik in TypScript leichter lesen. In gewissen Funktionen ist der Code in Python schlanker als in TypeScript, dafür lässt sich der Code in TypeScript besser wiederverwenden.

5.5 Zusammenfassung Kapitel 5

Die Portierung wird durch eine Kombination aus statischer Typprüfung und dynamischen Tests abgesichert. Der TypeScript-Compiler schließt große Teile typischer Python-Fehlerklassen bereits vor der Ausführung aus, während Unit-/Integrations- und Differentialtests die semantische Äquivalenz der übersetzten Beispiele praktisch validieren. Grenzen ergeben sich vor allem aus Laufzeitunterschieden (Equality/Hashing, Zahlenmodell, Konversionen), die im Projekt durch spezifische Datenstrukturen und explizite Guards adressiert werden.

6 Kritische Reflexion und Ausblick

Dieses Kapitel fasst die Ergebnisse der Studienarbeit zusammen und ordnet die Portierung der Python-Lehrbeispiele in TypeScript hinsichtlich Zielerreichung, Nutzen und Grenzen ein. Abschließend werden konkrete Erweiterungs- und Forschungsperspektiven für eine weitergehende, stärker automatisierte Übersetzung diskutiert.

6.1 Erreichte Ziele und zentrale Ergebnisse

Ziel des Projekts war es, die in der Lehrveranstaltung *Formale Sprachen* verwendeten, ausführbaren Python-Beispiele in eine TypeScript-basierte Umgebung zu überführen, ohne die fachliche Semantik der Beispiele zu verändern. Dabei standen zwei Aspekte im Vordergrund:

1. **Strukturelle Semantiktreue:** Die übersetzten Implementierungen sollen dieselben Konzepte und Algorithmen ausdrücken (z. B. Scanner-/Parser-Pipelines, AST-nahe Datenformen, Automatenkonstruktionen).
2. **Explizite Typisierung:** Implizite Python-Datenformen sollen in TypeScript als *statisch überprüfbare* Typen modelliert werden, sodass Inkonsistenzen frühzeitig erkannt werden und Schnittstellen präziser dokumentiert sind. [8, S.44ff]

Als Ergebnis liegt eine TypeScript-Codebasis vor, die die wesentlichen Lehrinhalte aus den Python-Notebooks in einer Umgebung mit statischer Typprüfung reproduzierbar bereitstellt. Der entscheidende Unterschied zur Python-Referenz besteht darin, dass viele zuvor nur informell angenommene Invarianten (z. B. “Parser liefert immer (Wert, Restliste)”, “Transitionstabelle hat wohldefinierte Schlüssel”) nun im Typmodell explizit abgebildet sind und damit bereits durch den Compiler überprüft werden.

6.2 Mehrwert der statischen Typisierung im Lehr- und Projektkontext

Die Portierung zeigt, dass TypeScript im Kontext compiler-naher Strukturen (Token, Parserzustände, ParseTrees/ASTs, Automaten) insbesondere dann Vorteile bietet, wenn Datenformen *rekursiv* sind oder wenn viele Teilkomponenten über klar definierte Schnittstellen interagieren. Der Mehrwert lässt sich in drei Punkten bündeln:

6.2.1 Frühzeitige Fehlererkennung und bessere Schnittstellen

Durch die Typprüfung werden Fehler, die in Python typischerweise erst bei der Ausführung sichtbar werden, bereits beim Übersetzen erkannt. Beispiele sind vertauschte Rückgabekomponenten, vergessene Fälle in Union-Typen oder inkonsistente Schlüsseltypen in Abbildungen. In diesem Sinne übernimmt das Typsystem eine Rolle als *Prüfinstanz*, die zulässige Zustände eingrenzt und damit das Risiko “schiefgehender” Programmpfade reduziert. [2, S.1ff]

6.2.2 Dokumentation durch Typen und Wartbarkeit

Typdefinitionen (insbesondere bei Union-/Tupel-Modellen) wirken als präzise Dokumentation. Für Lehrbeispiele ist dies didaktisch relevant: Studierende sehen nicht nur den Algorithmus, sondern auch dessen erwartete Datenformen explizit. Zudem werden spätere Erweiterungen (z. B. neue Tokenarten, zusätzliche AST-Formen) systematisch durch den Compiler begleitet, weil an betroffenen Stellen Typfehler auftreten, bis alle notwendigen Anpassungen erfolgt sind. [8, S.44ff]

6.2.3 Strukturelle Typisierung als Brücke zu Python-Datenmodellen

Da TypeScript strukturell typisiert ist, können viele Python-nahe Datenmodelle direkt als Records, Tupel und Unions modelliert werden, ohne eine nominelle Klassenhierarchie einzuführen. Das ist für die Portierung besonders hilfreich, weil die Python-Notebooks häufig *datenorientiert* (statt objektorientiert) formuliert sind. [8, S.44ff]

6.3 Einordnung der Transformation als compiler-nahe Übersetzung

Die in Kapitel 4 beschriebene Transformation lässt sich konzeptionell als vereinfachte, syntaxgerichtete Übersetzung verstehen, bei der aus einer Quellrepräsentation schrittweise eine Zielrepräsentation erzeugt wird. In compiler-theoretischer Terminologie entspricht dies einer Kombination aus: (i) syntaktischer Analyse (Parser), (ii) semantischer Modellierung/Transformation (AST-Transformation) und (iii) Codegenerierung. [7, S.304ff]

Die praktische Portierung ist dabei weniger als *vollautomatische Übersetzung* zu verstehen, sondern als *systematische Rekonstruktion* der Programme unter Beibehaltung der algorithmischen Kernaussage. Die wichtigste Zusatzleistung entsteht nicht durch neue Algorithmen, sondern durch die explizite Formalisierung zuvor impliziter Python-Datenformen in Form von TypeScript-Typen.

6.4 Grenzen der Portierung

Trotz der erzielten Semantiktreue bestehen Grenzen, die aus den unterschiedlichen Laufzeit- und Sprachmodellen resultieren:

- **Python-spezifische Dynamik:** Python erlaubt sehr flexible Datenformen (z. B. heterogene Listen, spontane Strukturänderungen, “duck typing” zur Laufzeit). Eine 1:1-Übernahme dieser Flexibilität ist in TypeScript entweder nicht sinnvoll oder würde zur Verwässerung des Typmodells (z. B. durch `any`) führen. [8, S.44ff]
- **Unterschiede im Zahlenmodell:** Python unterscheidet `int` und `float`, TypeScript fasst dies in `number` zusammen. Semantisch problematisch wird dies bei sehr großen Ganzzahlen oder exakter Ganzzahlarithmetik.
- **Gleichheit/Schlüsselstrukturen:** Python kann komplexe, unveränderliche Strukturen (z. B. `frozenset`, `tuple`) als Schlüssel in `dict` verwenden. In TypeScript muss diese Semantik durch geeignete Container-/Schlüsselrepräsentationen explizit nachgebildet werden, da `Map/Set` referenzbasiert arbeiten.

Diese Grenzen sind nicht als Mängel der Portierung zu verstehen, sondern als Konsequenz der unterschiedlichen Designziele beider Sprachökosysteme. In der Arbeit wurde daher konsequent priorisiert, *Typpräzision und Semantik* zu erhalten, statt Python-Dynamik um jeden Preis zu simulieren.

6.5 Ausblick: Technische Erweiterungen und Forschungsfragen

Aus dem Projekt ergeben sich mehrere naheliegende Erweiterungen, die sowohl didaktisch als auch technisch relevant sind:

6.5.1 (Teil-)Automatisierte Typableitung und Übersetzung

Ein nächster Schritt wäre die stärkere Automatisierung der Übersetzung: Anstatt Typabbildungen manuell zu rekonstruieren, könnten Regeln für die Ableitung von TypeScript-Typen aus Python-Annotationen und aus beobachteten Datenformen systematisiert werden. In compiler-naher Perspektive entspricht dies einer Erweiterung der “semantischen Regeln” der Übersetzung. [7, S.304ff]

6.5.2 Systematische Korrektheitsabsicherung

Kapitel 5 validiert die Semantik primär empirisch (Tests, Vergleichsausführungen). Als Ausblick bietet sich an, Teilaspekte formaler zu fassen, etwa für ausgewählte

Kernfunktionen:

$$\forall P \in \mathcal{P} : \llbracket P \rrbracket_{\text{Python}} = \llbracket f(P) \rrbracket_{\text{TS}}$$

In der Praxis könnte dies durch stärkeres Property-based-Testing oder durch formale Spezifikationen einzelner Komponenten angenähert werden. [7, S.15ff]

6.5.3 Didaktische Aufbereitung als Toolchain

Langfristig kann die TypeScript-Version als eigenständige Toolchain für Lehrzwecke ausgebaut werden (z. B. als NPM-Paket oder Web-Demo), sodass Studierende direkt in einer einheitlichen Umgebung experimentieren können. Besonders attraktiv ist dabei die Möglichkeit, Typfehler als unmittelbares Feedback in IDEs zu nutzen (Autocomplete, Refactoring, statische Hinweise).

6.5.4 Erweiterung des Sprachumfangs

Die übersetzten Beispiele decken einen klaren Ausschnitt ab (formale Sprachen, Parsing, Automaten, AST-nahe Ausdrücke). Eine Erweiterung auf weitere Python-Konstrukte (z. B. komplexere Ausdrucksformen, zusätzliche Datentypen, mehr Standardbibliothek) müsste bewusst abwägen, welche Dynamik sinnvoll typisierbar ist und wo die Domäne eher eine abstrahierte Modellierung verlangt.

6.6 Schlussbemerkung

Die Arbeit zeigt, dass die Portierung von Python-Lehrbeispielen in TypeScript nicht nur eine syntaktische Übertragung ist, sondern eine *Modellierungsaufgabe*: Implizite Datenformen werden explizit, informelle Invarianten werden zu Typen, und die Compilerprüfung wird zu einem integralen Teil der Qualitätssicherung. Damit entsteht eine robuste Grundlage, auf der die behandelten Inhalte aus dem Bereich formaler Sprachen konsistent, wartbar und didaktisch nutzbar bereitgestellt werden können. [8, S.44ff]

7 Literaturverzeichnis

- [1] Yilin Lu, Shurui Du und Qianwen Wang. „THE AGENTOPIA TIMES: Understanding and Mitigating Hallucinations in Multi-Agent LLM Systems via Data Journalism Gameplay“. In: *2025 IEEE Visualization and Visual Analytics (VIS)*. IEEE, 2025, S. 156–160. DOI: 10.1109/VIS60296.2025.00037.
- [2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN: 978-0262162098.
- [3] *The TypeScript Handbook*. TypeScript Team (Microsoft). 2026. URL: <https://www.typescriptlang.org/docs/handbook/2/narrowing.html> (besucht am 08.02.2026).
- [4] John E. Hopcroft, Rajeev Motwani und Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3. Aufl. Pearson, 2006. ISBN: 978-0321455369.
- [5] Dick Grune und Criel J. H. Jacobs. *Parsing Techniques: A Practical Guide*. 2. Aufl. Springer, 2008. ISBN: 978-0387202488.
- [6] *Lezer Documentation*. CodeMirror / Lezer Project. 2026. URL: <https://lezer.codemirror.net/docs/> (besucht am 08.02.2026).
- [7] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools*. 2. Aufl. Addison-Wesley, 2006. ISBN: 978-0321486813.
- [8] Dan VanderKam. *Effective TypeScript: 62 Specific Ways to Improve Your TypeScript*. O'Reilly Media, 2019. ISBN: 978-1492053743.

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.